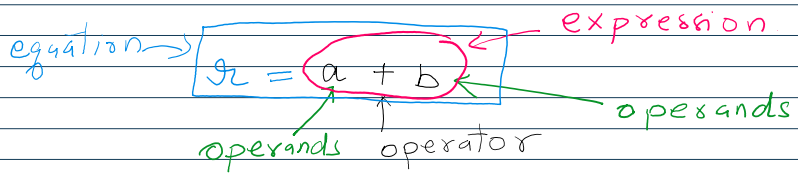
Operators in Java

12 April 2025 09:42

Operators are the symbols which denotes the type of operation.



(a + b) = (r) ✓ math
✗ computer programming

result = expression
will store on left hand side

Different types of operators in Java:

- Unary Operator
- Binary Operator
- Ternary Operator.

Unary operator :- It uses one operator to perform its operation.

Types of unary operator

- Unary (+), Unary (−) → These operators are used to represent sign magnitude of a numeric value

For example. → operand

```
int x = -5;
        ↑
      unary (-)
int y = -x;
int z = +10;
        ↓
      unary (+)
```

- increment operator (++)
- decrement operator (--)

increment operator : This operator can increments the variable value by '1', always.

For example:

```
int x = 5;
x
5   (int)
x++;  ——> Postfix
```

```
x
5   (int)
x++; ——> Postfix
x
6   (int)
x++;
x
7   (int)
```

Increment operator in Java can be applied in two ways.

Postfix: In postfix we write the operator after the operand. In postfix expression the variable current value is used in the expression, after completion of expression, the variable value is updated by one or '1'.

For example:

```
x
7   (int)
int r = x++ - 3;
r
4   (int)
x
8   (int)
```

Prefix: In prefix, we write the operator before the operand. In prefix expression the variable is updated first, and then its value is used in the expression.

For example:

```
x
8   (int)
r = ++x + 5;
r
14   (int)
x
9   (int)
```

'x' is updated first, and then it is used in the expression.

Decrement Operator: The decrement operator decrement or reduces the value of variable by '1'.

Postfix: In postfix we write the operator after the operand. In postfix expression the variable current value is used in the expression, after completion of expression, the variable value is updated by one or '1'.

For example:

```
x
9   (int)
x--;
x
8   (int)
```

——> decrementing the value of 'x' by '1'.

```
x--;
x
8   (int)
r = x-- - 4;
r
4   (int)
x
7   (int)
```

→ decrementing the value of 'x' by '1'.

→ the value of x is decrement by '1' after the completion of the expression.

Prefix: In prefix, we write the operator before the operand. In prefix expression the variable is updated first, and then its value is used in the expression.

For example:

```
x
7   (int)
--x;
x
6   (int)
r = --x + 5;
r
10   (int)
x
5   (int)
```

→ The value of 'x' is reduced by '1' first and then it is used in the expression

Precedence of operators

Exponent → BODMAS

Add

PEDMAS → Subtract

Divide, Multiply

Parentheses '()'

Operator Precedence

Operators	Precedence
postfix	expr++ expr--
unary	++expr --expr +expr -expr ~ !
→ multiplicative	* / % (remainder)
additive	+ -
→ shift	<< >> >>>
→ relational	< > <= >= instanceof
→ equality	== !=
→ bitwise AND	&
→ bitwise exclusive OR	^
→ bitwise inclusive OR	|
→ logical AND	&&
→ logical OR	||
→ ternary	? :
→ assignment	= += -= *= /= %= &= ^= |= <<= >>= >>>=

```
x
5   (int)
r = x++ - --x;
r
0   (int)
```

5 → prefix

before executing this postfix will update the variable value.

postfix (higher precedence) than prefix.

x = 6 after checking all the operators used in the exper.

then '--x' will execute.

x = 5

The current value 'x = 5'.

Binary Operators :- It uses two operands to perform its operation.

⇒ a + b * c + d / e

higher, higher

⇒ Arithmetic operators : *, /, % (modulus or remainder)

⇒ Arithmetic operators : *, /, % (modulus or remainder)

+, −

⇒ Relational operators : It always results in boolean (true or false)

>, <, >=, <=, ==, !=

(== : equality operator)

Example of relational operators :

```
r
0   (int)
x
2   (int)
y
5   (int)
x > y
false   (boolean)
x < y
true   (boolean)
r == 0
true   (boolean)
r > y
false   (boolean)
```

⇒ Assignment operator : It is used to assign values from right side to its left.

'=', '+=', '-=', '*=', '/=', '%='

Example :

```
x
5   (int)
x += 2;
x
7   (int)
x -= 3;
x
4   (int)
x *= 2;
x
8   (int)
x /= 3;
x
2   (int)
x %=3;
x
2   (int)
```

Logical operators : ! (NOT), || (OR), && (AND)

both condition must be 'true' to get the result as 'true', otherwise 'false'.

! : if the result is 'true' then it is converted to 'false', otherwise vice versa

|| : if any condition is 'true' the result is 'true', to get 'false' in OR, both condition must be 'false'.

'false', otherwise vice versa

to get 'false' in OR, both condition must be 'false'.

For example:

```
x
2   (int)
y
5   (int)
r
0   (int)
!(x > 0)
false   (boolean)
!(x < 0)
true   (boolean)
x > 0 && r == 0
true   (boolean)
x > 0 && r  > 0
false   (boolean)
x > 0 && r  >= 0
true   (boolean)
```

⇒ Shift operators
⇒ bitwise operator } "I do it later"

What is Syntax?

Ans:- Grammatical rule followed in programming language.

ternary Operator, conversion between data-types

Ternary Operator: It require 3 operands to do its operation.

Syntax:

variable = (<condition>) ? statement 1 : statement 2 ;

True → value assigned to the variable

False → value assigned to a variable.

When condition is 'true' statement 1 will be followed, otherwise statement two will be followed

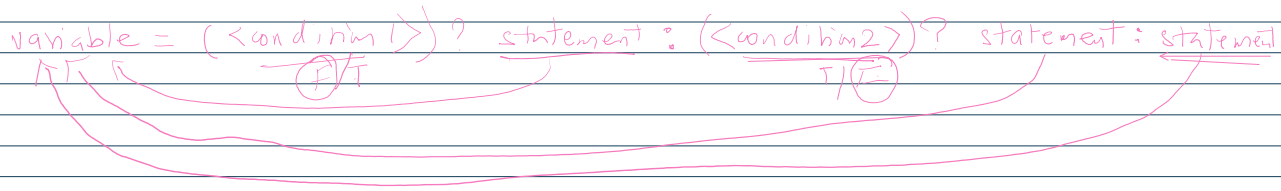
For example:

```
int x = 20;
int y = 30;
int z = 45;
int r = (x > y)? x:y;
r
30   (int)
r = (x > y || y < z)? x:y;
r
20   (int)
```

The ternary operator can be nested

Syntax:

variable = (<condition1>) ? (<condition2) ? ... : (<condition N1)? ... ;



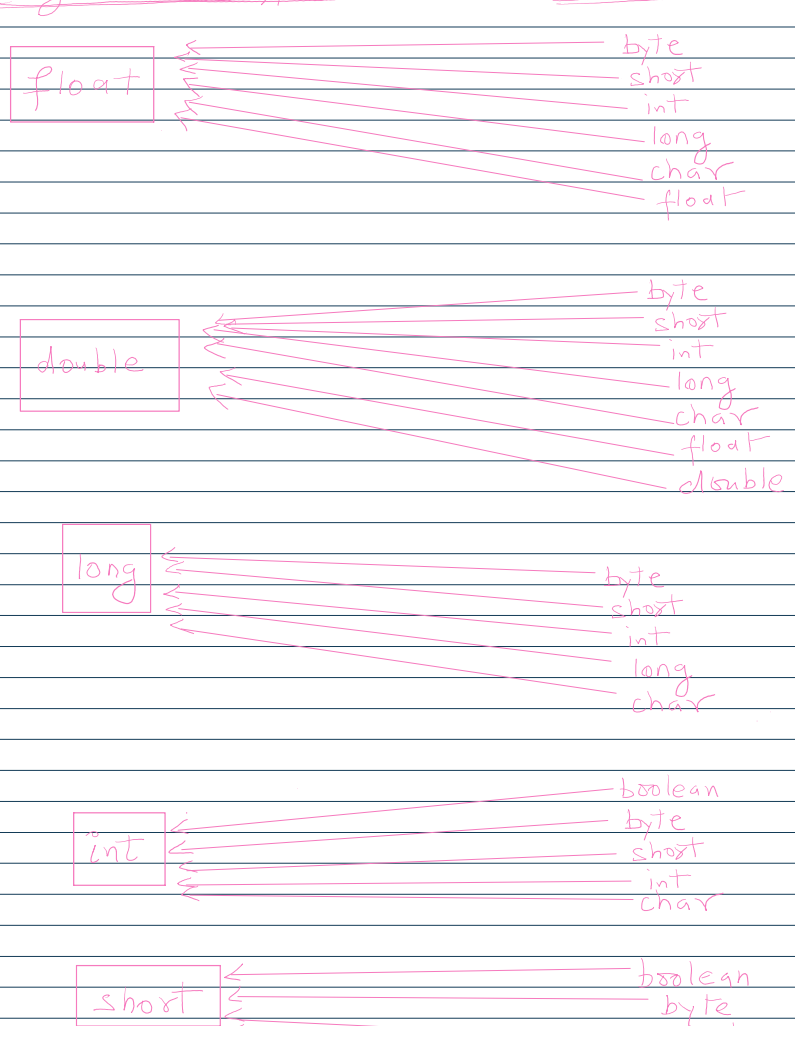
For example:

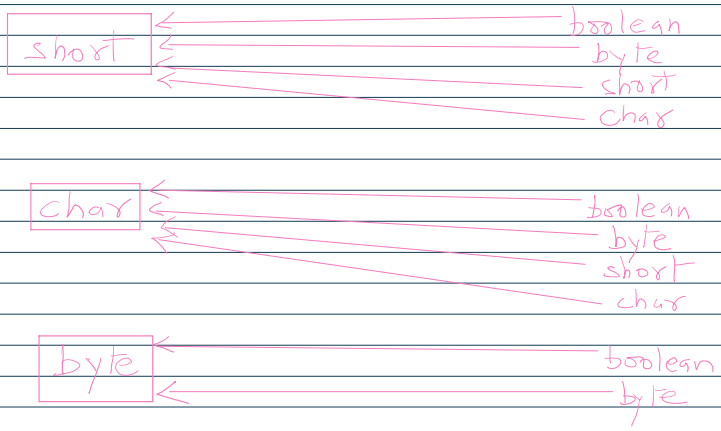
```
x
20   (int)
y
30   (int)
z
45   (int)
int max = (x > y && x > z)? x: (y > z)? y: z;
max
45   (int)
```

Conversion between data-types

We can convert one data type to another data, by using implicit type casting or explicit type casting.

Implicit type casting is done automatically by Java compiler. And, there no data loss. It converts lower data type to higher datatype





For example:

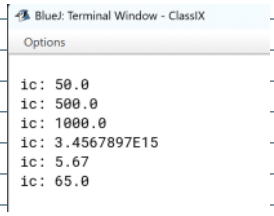
```
package Chapter0;


/**
 * Write a description of class ImplicitConversion here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ImplicitConversion
{
    public static void main(String args[]){
        boolean b = true;
        byte bi = 50;
        short s = 500;
        int i = 1000;
        long l = 3456789765445678L;
        float f = 5.67f;
        double d = 7.89;
        char c = 'A';

        float ic;
        //ic = b; //not possible
        double id;
        //id = b; //not possible
        long li;
        // li = b; //not possible
        // boolean type cannot be converted to any other type
        // either explicitly or implicitly

        //In the HW try to convert boolean to other types.
        System.out.print("\f"); //always fresh terminal window
        ic = bi;
        System.out.print("\n ic: "+ic);
        ic = s;
        System.out.print("\n ic: "+ic);
        ic = i;
        System.out.print("\n ic: "+ic);
        ic = l;
        System.out.print("\n ic: "+ic);
        ic = f;
        System.out.print("\n ic: "+ic);
        //ic = d; implicit type casting is not possible in this case, data type of 'd'
        // is double.
        ic = c;
        System.out.print("\n ic: "+ic);
    }
}
```

Output:



Home work: Try out other implicit conversions in the bluej, and update the code.

parameter input

public static void main(int x) {

This is not the standard way to write the code.

public static void main(String args[])

Primitive type | Wrapper class type

Primitive type	Wrapper class type
boolean	Boolean
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character

all of these class contain convertible data type methods

it creates number of memory cells as per our requirement, but stores only String type.



Therefore we need to convert String type to our required type.

standard function or method recognized by JVM.

parseDataTypeToConvert(String s) method used to convert String to required type.

```
public class UsingArgsParameter{
    public static void main(String[] args) {
        int x = Integer.parseInt(args[0]);
        double d = Double.parseDouble(args[1]);
        System.out.println("x = " + x);
        System.out.println("d = " + d);
        main(x);
    }
    public static void main(int x) { //not the standard function recognized by JVM
        System.out.println("x = " + x);
    }
}
```

Output:

```
PS C:\Users\bluej\Documents\Students\KanavPasari\coding\ClassIXVSC\Chapter1> javac UsingArgsParameter.java
PS C:\Users\bluej\Documents\Students\KanavPasari\coding\ClassIXVSC\Chapter1> java UsingArgsParameter 10 3.14
x = 10
d = 3.14
x = 10
```

H.W Unsolved Java programs: Chapter-5.

Read the chapter & do communicate on doubts.